

Lista de exercícios

– Análise de algoritmos –

- Um algoritmo com tempo de execução, no pior caso, de $T(n) = cn^2 + 30$, leva $T(n)$ segundos para processar uma entrada de tamanho n . No caso de uma entrada de tamanho $n = 100$ tem-se o tempo $T(n) = 50$ segundos. Qual será o tempo para processar uma entrada de tamanho $n = 200$?
- Qual é a complexidade, utilizando a notação O , do algoritmo mencionado na questão anterior?
- A lista abaixo enumera o custo computacional, no pior caso, de diferentes funções. Determine sua complexidade usando a notação O .
 - $C(n) = 5 + 0,001n^3 + 0,025n$
 - $C(n) = 500n + 100n^{1,5} + 50n\log_5 n$
 - $C(n) = 0,5n + 5n^{1,5} + 2.5n^{1.75}$
 - $C(n) = n^2\log_2 n + n(\log_2 n)^2$
 - $C(n) = 100n + 0,01n^2$
 - $C(n) = 0,01n + 100n^2$
 - $C(n) = 2n + n^{0,5} + 0,5n^{1,25}$
 - $C(n) = 0,01n\log_2 n + n(\log_2 n)^2$
 - $C(n) = 100n\log_3 n + n^3 + 100n$
- Determine a função de custo $T(n)$ e a complexidade assintótica para cada uma das funções a seguir.

Observações:

- Assuma que cada linha executável possui custo $O(1)$.
- Considere o *pior caso*, quando aplicável
- Instruções do tipo `for(int i=1;i<=n;i++)` são executadas n vezes.

(a)

```
1 void func1(int n) {
2     int soma = 0;
3     for (int i = 0; i < n; i++) {
4         soma = soma + i;
5     }
6 }
```

(b)

```
1 void func2(int n) {
2     for (int i = 0; i < n; i++) {
3         printf("%d\n", i);
4     }
5     for (int j = 0; j < n; j++) {
6         printf("%d\n", j);
7     }
8 }
```

(c)

```
1 void func3(int n) {
2     for (int i = 0; i < n; i++) {
3         for (int j = 0; j < n; j++) {
4             printf("%d\n", i + j);
5         }
6     }
7 }
```

(d)

```
1 void func4(int n) {
2     for (int i = 0; i < n; i++) {
3         for (int j = 0; j <= i; j++) {
4             printf("%d\n", i + j);
5         }
6     }
7 }
```

(e)

```
1 void func5(int n) {
2     for (int i = 1; i < n; i = i * 2) {
3         printf("%d\n", i);
4     }
5 }
```

(f)

```
1 void func6(int n) {
2     while (n > 1) {
3         n = n / 2;
4     }
5 }
```

(g)

```
1 void func7(int n) {
2     for (int i = 0; i < n; i++) {
3         for (int j = 0; j < i * i; j++) {
4             printf("%d\n", j);
5         }
6     }
7 }
```

(h)

```
1 void func8(int n) {  
2     if (n % 2 == 0) {  
3         for (int i = 0; i < n; i++) {  
4             printf("%d\n", i);  
5         }  
6     } else {  
7         printf("impar\n");  
8     }  
9 }
```

Respostas:

- Exercício 1: O tempo de execução do algoritmo é dado por $T(n) = cn^2$. Para calcular o tempo de execução, além de saber o tamanho da entrada, deve-se saber o valor da constante c . Esse valor pode ser calculado da A partir dessa expressão, tem-se

$$\begin{aligned}T(n) &= cn^2 + 30 \\ 50 &= c \cdot 100^2 + 30 \\ 50 - 30 &= c \cdot 100^2 \\ 20/10000 &= c \\ c &= 0,002\end{aligned}$$

Sabendo-se o valor de c , pode-se calcular o tempo para a entrada de tamanho $n = 200$:

$$\begin{aligned}T(n) &= 0,002 \cdot 200^2 + 30 \\ &= 110 \text{ segundos}\end{aligned}$$

- Exercício 2: Se a complexidade é dada por $T(n) = cn^2 + 30$, deve-se desprezar (i) os termos de menor ordem (nesse caso, 30) e (ii) as constantes (nesse caso, c). Assim, a complexidade é $O(n^2)$.

- Exercício 3

- $O(n^3)$
- $O(n^{1.5})$
- $O(n^{1.75})$
- $O(n^2 \log n)^1$
- $O(n^2)$
- $O(n^2)$
- $O(n^1, 25)$
- $O(n(\log n)^2)$
- $O(n^3)$

- Exercício 3

- Linha 2: uma execução
Linha 3: n execuções
Linha 4: n execuções

Logo:

- $T(n) = 2n + 1$
- Complexidade: $O(n)$

- Linha 2: n execuções
Linha 3: n execuções
Linha 5: n execuções
Linha 6: n execuções

Logo:

- $T(n) = 4n$
- Complexidade: $O(n)$

- Linha 2: n execuções
Linha 3: n^2 execuções
Linha 4: n^2 execuções

Logo:

¹Na notação O , assume-se $\log n = \log_2 n$

- $T(n) = 2n^n + n$
- Complexidade: $O(n^2)$
- d. Linha 2: n execuções
 Linha 3: $\sum_{x=1}^n x = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$
 Linha 4: $\frac{n^2+n}{2}$
 Logo:
 – $T(n) = n + 2 \left(\frac{n^2+n}{2} \right) = n^2 + 2n$
 – Complexidade: $O(n^2)$
- e. Linha 2: $\lceil \log_2 n \rceil$ execuções
 Linha 3: $\lceil \log_2 n \rceil$ execuções
 Observações:
 — Para k iterações, $i = 1, 2, 4, 8, \dots, 2^k$
 — O laço *for* executa enquanto $2^k < n$
 Logo:
 – $T(n) = \lceil \log_2 n \rceil + \lceil \log_2 n \rceil = 2(\lceil \log_2 n \rceil)$
 – Complexidade: $O(\log_2 n)$
- f. Em k iterações, n assume os seguintes valores: $n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, \frac{n}{2^k}$
 A linha 2 é executada até que $\frac{n}{2^{k-1}} = 1$.
 Ou seja, a linha 2 é executada até a k^{a} iteração, em que $2^{k-1} = n$.
 Para saber quantas vezes a linha 2 é executada, temos que descobrir o k .
 Se, na última iteração, $2^{k-1} = n$, então $k-1 = \log_2 n$ e $k = \log_2 n + 1$
 Ou seja:
 – A linha 2 é executada $\log_2 n + 1$ vezes.
 – A linha 3 é executada $\log_2 n$ vezes;
 Logo:
 – $T(n) = \log_2 n + \log_2 n + 1 = 2(\log_2 n) + 1$
 – Complexidade: $O(\log_2 n)$
- g. A linha 2 é executada n vezes ($i \in \{0, \dots, n-1\}$). Ou seja, o laço externo é executado n vezes.
 A cada execução do laço externo:
 – a linha 3 é executada i^2 vezes ($j \in \{0, \dots, i^2\}$)
 – a linha 4 é executada i^2 vezes
 Execuções da linha 3: $\sum_{i=0}^{n-1} i^2 = \frac{n(n-1)(2n-1)}{6} = \frac{2n^3-3n^2+n}{6}$
 Execuções da linha 4: $\sum_{i=0}^{n-1} i^2 = \frac{n(n-1)(2n-1)}{6} = \frac{2n^3-3n^2+n}{6}$
 Logo:
 – $T(n) = n + 2 \left(\frac{2n^3-3n^2+n}{6} \right) = n + \frac{4n^3-6n^2+2n}{6}$
 – Complexidade: $O(n^3)$
- h. Linha 2: 1 execução
 Linha 3: $n+1$ execuções
 Linha 4: n execuções
 Logo:
 – $T(n) = 1 + n + 1 + n = 2n + 2$
 – Complexidade: $O(n)$